Day 15 : JWT Authentication & API Setup

1. Introduction to JWT Authentication

Authentication is the process of verifying a user's identity before allowing access to an application.

Examples:

- Login with Email and Password
- Login with Google
- Login with OTP

Without authentication, anyone could access protected data.

2. What is JWT?

JWT (JSON Web Token) is a secure token generated by the server after successful login.

Instead of sending username and password every time, the app sends the JWT token.

Structure :

```
xxxxx.yyyyy.zzzzz
```

1. Header
   Contains token type and algorithm

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. **Payload**
   Contain User information

```
{
  "id": 1,
  "email": "abc@gmail.com"
}
```

3. Signature
   Used to verify token authenticity.

3. Why JWT??

Advantages:

- Secure
- Lightweight
- Stateless
- Fast API communication

Used in:

- Mobile Apps
- Web Applications
- REST APIs

4. Authentication Flow?

```
User Login
     ↓
Email + Password
     ↓
API Request
     ↓
Server Verification
     ↓
JWT Token Returned
     ↓
Store Token Securely
     ↓
Navigate Home
     ↓
Use Token for API Calls
```

4. Required Packages

1, HTTP Package

Used for API communication

```
dependencies:
  http: ^1.0.0
```

Functions:

- GET
- POST
- PUT
- DELETE

6. Dio Package

Alternative to HTTP

```
dependencies:
  dio: ^5.0.0
```

7 . Flutter Secure Storage

Stores sensitive data securely.

```
dependencies:
  flutter_secure_storage: ^9.0.0
```

8. API Service

Purpose:

- Handle all API calls
- Manage Base URL
- Manage Headers
- Handle Errors

```
class ApiService {
  static const String baseUrl =
      "https://api.example.com";
}
```

9. Login API

When user clicks Login:

```
Password
    ↓
POST Request
    ↓
Server Verification
    ↓
JWT Token Returned
```

Example Request :

```
{
  "email":"abc@gmail.com",
  "password":"123456"
}
```

Example Response

```
{
  "token":"abcxyz123"
}
```

10. Store Token Securely

```
await storage.write(
  key: "token",
  value: token,
);
```

11. Registration API

```
User Details
     ↓
Validation
     ↓
Register API
     ↓
Account Created
     ↓
Auto Login
```

Example Request :

```
{
 "name":"Sneha",
 "email":"abc@gmail.com",
 "password":"123456"
}
```

12. Validation

Before API call:

Email Verification :

```
abc@gmail.com ✓
abc@gmail ✗
```

13. Auto Login

On app start:

```
String? token =
    await storage.read(key: 'token');
```

If token exists: Home Screen
Else : Login screen

Logout

Steps:

1. Delete token
2. Navigate to Login

```
await storage.delete(key: 'token');
```

Output :